实验　软件架构

一、实验目的

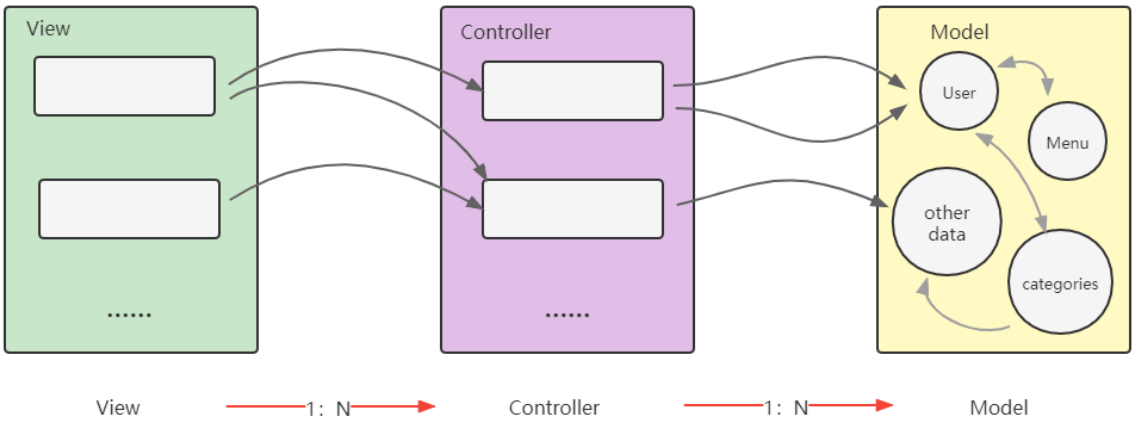
1 熟悉前后端分离下的软件架构。
2 实现前后端通信
3 实现后端与数据库通信
4 完成前后端分离实验《Todos》

二、实验装置

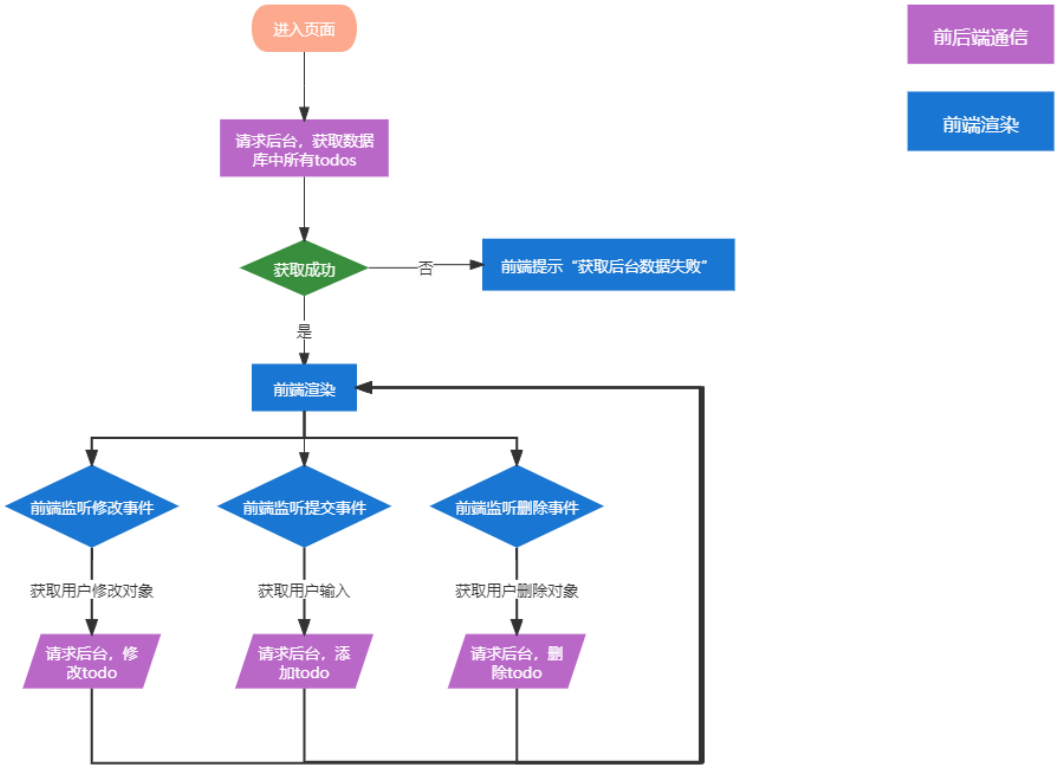
IDE: VS Code
后端（Java）：jdk8, jre8
后端（node）: node, Npm
数据库: MySQL, navicat
前端: Yarn

三、实验内容

1. MVC 架构



2. 系统流程



3. 后端连接数据库

3.1 配置文件

进入 be-spring/src/main/resources，创建 application.properties

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3306/todos
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name =com.mysql.jdbc.Driver
```

3.2 定义数据访问层 Data Access Object

```
package com.example.accessingdatamysql;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccessingDataMysqlApplication {
  public static void main(String[] args) {
    SpringApplication.run(AccessingDataMysqlApplication.class, args);
  }
}
```

3.3 定义数据模型 Model

进入 be-spring/src/main/java/com/example/accessingdatamysql/model，创建 TodoList.java

```
package com.example.accessingdatamysql;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity // This tells Hibernate to make a table out of this class
public class TodoList {
  @Id
//  @GeneratedValue(strategy=GenerationType.AUTO)
  private String id;

  private String text;

  private Integer finished;

  public String getId() {
    return id;
  }

  public void setId(String id) {
    this.id = id;
  }

  public String getText() {
    return text;
  }

  public void setText(String text) {
    this.text = text;
  }

  public Integer getFinished() {
    return finished;
  }

  public void setFinished(Integer finished) {
    this.finished = finished;
```

```
  }
}
```

4. 后端操作数据库

4.1 查询数据库中所有 todos

```
public @ResponseBody Iterable<TodoList> getAllTodos() {
    // This returns a JSON or XML with the todos
    return todoListRepository.findAll();
  }
```

4.2 往数据库里增加一条 todo

```
public @ResponseBody String addNewTodo (@RequestBody String todo) {
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    TodoList newTodo = new TodoList();
    newTodo.setId(todoMap.get("id").toString());
    newTodo.setText(todoMap.get("text").toString());
    newTodo.setFinished(0);
    todoListRepository.save(newTodo);
    return "Saved";
  }
```

4.3 修改数据库中 todo 的 finish 状态

```
public @ResponseBody String fininshTodo (@RequestBody String todo) {
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    TodoList newTodo =
todoListRepository.findById(todoMap.get("id").toString()).get();
    newTodo.setFinished("true".equals(todoMap.get("finished").toString()
) ? 1 : 0);
    todoListRepository.save(newTodo);
    return "Saved";
  }
```

4.4 根据 id 删除数据库中的一条 todo

```
public @ResponseBody String deleteTodo (@RequestBody String todo) {
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    todoListRepository.deleteById(todoMap.get("id").toString());
    return todoMap.get("id").toString();
```

```
  }
```

4.5 修改为 rest 风格

```
@RequestMapping(path="/api")
    @GetMapping(path="/getTodos")
    @PostMapping(path="/addTodo",
produces="application/json;charset=UTF-8")
    @PostMapping(path="/finishTodo",
produces="application/json;charset=UTF-8")
    @PostMapping(path="/delTodo",
produces="application/json;charset=UTF-8")
```

4.6 完整的 Controller

```
package com.example.accessingdatamysql;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RequestBody;
import com.alibaba.fastjson.JSON;
import java.util.Map;


@Controller // This means that this class is a Controller
@RequestMapping(path="/api") // This means URL's start with /api (after
Application path)
public class MainController {
  @Autowired // This means to get the bean called todoListRepository
Which is auto-generated by Spring, we will use it to handle the data
  private TodoListRepository todoListRepository;

  @GetMapping(path="/getTodos")
  public @ResponseBody Iterable<TodoList> getAllTodos() {
    // This returns a JSON or XML with the todos
    return todoListRepository.findAll();
  }

  @PostMapping(path="/addTodo", produces="application/json;charset=UTF-
8") // Map ONLY POST Requests
  public @ResponseBody String addNewTodo (@RequestBody String todo) {
```

```
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    TodoList newTodo = new TodoList();
    newTodo.setId(todoMap.get("id").toString());
    newTodo.setText(todoMap.get("text").toString());
    newTodo.setFinished(0);
    todoListRepository.save(newTodo);
    return "Saved";
  }

  @PostMapping(path="/finishTodo",
produces="application/json;charset=UTF-8")
  public @ResponseBody String fininshTodo (@RequestBody String todo) {
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    TodoList newTodo =
todoListRepository.findById(todoMap.get("id").toString()).get();
    newTodo.setFinished("true".equals(todoMap.get("finished").toString()
) ? 1 : 0);
    todoListRepository.save(newTodo);
    return "Saved";
  }

  @PostMapping(path="/delTodo", produces="application/json;charset=UTF-
8")
  public @ResponseBody String deleteTodo (@RequestBody String todo) {
    Map<String, Object> todoObj = JSON.parseObject(todo, Map.class);
    Map<String, Object> todoMap =
JSON.parseObject(todoObj.get("data").toString(), Map.class);
    todoListRepository.deleteById(todoMap.get("id").toString());
    return todoMap.get("id").toString();
  }
}
```

5 前端与后端通信

5.1 发起添加一条 todo 的请求

```
// post to backend
      axios.post(`/${useApi}/addTodo`, {
        headers: {'Content-Type': 'application/json'},
        data: JSON.stringify(todoObj)
      }).then((res) => {
```

```
        // request from backend
        if (res.status === 200) {
          // update localStorage & page
          setLocalTodos(
            [
              ...localTodos,
              todoObj
            ]
          )
        } else {
          throw new Error(res)
        }
      }).catch((err) => {
        throw new Error(err)
      })
```

5.2 发起修改一条 todo 的 finish 状态的请求

```
// post to backend
    axios.post(`/${useApi}/finishTodo`, {
      headers: {'Content-Type': 'application/json'},
      data: JSON.stringify(todoObj)
    }).then((res) => {

      // request from backend
      if (res.status === 200) {
        // update localStorage & page
        setLocalTodos(localTodos.map(localItem => {
          if (localItem.id === todoItem.id) localItem.finished
= !localItem.finished
          return localItem
        }))
      } else {
        throw new Error(res)
      }
    }).catch((err) => {
      throw new Error(err)
    })
```

5.3 发起删除一条 todo 的请求

```
axios.post(`/${useApi}/delTodo`, {
        headers: {'Content-Type': 'application/json'},
```

```
        data: JSON.stringify(todoObj)
      }).then((res) => {

        // request from backend
        if (res.status === 200) {
          // update localStorage & page
          setLocalTodos(localTodos.filter(item => item.id !== id))
        } else {
          throw new Error(res)
        }
      }).catch((err) => {
        throw new Error(err)
      })
```

5. 4 完整的前端

```
import { useState, useRef, useEffect } from 'react';
import axios from 'axios';
import './App.css';

// set 'api1' for egg.js || set 'api2' for spring boot
const useApi = 'api2'

function App() {
  const input = useRef()
  const [localTodos, setLocalTodos] =
useState(localStorage.getItem('todos') ?
JSON.parse(localStorage.getItem('todos')) : []) // null -> []
  const [clickFreeze, setClickFreeze] = useState(false)

  // component did mount
  useEffect(()=>{
    axios.get(`/${useApi}/getTodos`).then((res) => {
      const data = res.data
      console.log(`data from ${useApi}`, data)
      setLocalTodos(data)
    }).catch((err) => {
      console.log(err)
      window.alert('获取后台数据失败')
    })
  }, [])

  // state update
  useEffect(() => {
    localStorage.setItem('todos', JSON.stringify(localTodos))
```

```
}, [localTodos])

// enter
const addTodo = (event) => {
  event.preventDefault()
  const inputVal = input.current.value

  // check in fontend, input can not be empty
  if (inputVal) {
    const todoObj = {
      'id': generateId().toString(),
      'text': inputVal.toString(),
      'finished': false
    }
    // post to backend
    axios.post(`/${useApi}/addTodo`, {
      headers: {'Content-Type': 'application/json'},
      data: JSON.stringify(todoObj)
    }).then((res) => {

      // request from backend
      if (res.status === 200) {
        // update localStorage & page
        setLocalTodos(
          [
            ...localTodos,
            todoObj
          ]
        )
      } else {
        throw new Error(res)
      }
    }).catch((err) => {
      throw new Error(err)
    })

    // clear input
    input.current.value = ''
  } else {
    window.alert('invalid input!')
  }
}

// left click
```

```
  const finishTodo = (todoItem) => {
    const todoObj = {
      id: todoItem.id,
      finished: !todoItem.finished
    }

    // post to backend
    axios.post(`/${useApi}/finishTodo`, {
      headers: {'Content-Type': 'application/json'},
      data: JSON.stringify(todoObj)
    }).then((res) => {

      // request from backend
      if (res.status === 200) {
        // update localStorage & page
        setLocalTodos(localTodos.map(localItem => {
          if (localItem.id === todoItem.id) localItem.finished
= !localItem.finished
          return localItem
        }))
      } else {
        throw new Error(res)
      }
    }).catch((err) => {
      throw new Error(err)
    })


  }

  // right click
  const delTodo = (event, id) => {
    event.preventDefault()
    if (clickFreeze) {
      window.alert('click freezing')
    } else {
      // freeze click
      setClickFreeze(true)
      setTimeout(() => {
        // thaw click
        setClickFreeze(false)
      }, 500);

      const todoObj = { id }
```

```
      axios.post(`/${useApi}/delTodo`, {
        headers: {'Content-Type': 'application/json'},
        data: JSON.stringify(todoObj)
      }).then((res) => {

        // request from backend
        if (res.status === 200) {
          // update localStorage & page
          setLocalTodos(localTodos.filter(item => item.id !== id))
        } else {
          throw new Error(res)
        }
      }).catch((err) => {
        throw new Error(err)
      })
    }


  }

  // generate random id which consisted of lowercase & uppercase &
number
  const generateId = () => {
    return Date.now()%10000000 +
String.fromCharCode(Math.floor(Math.random() * 26) + 97) +
String.fromCharCode(Math.floor(Math.random() * 26) + 65)
  }

  return (
    <div className="App">
      <h1>todos</h1>
        <form id="form" onSubmit = {e => addTodo(e)}>
          <input id="input" ref={input} type="text" placeholder="Enter
your todo" />
          <ul id="todos">
            {
              localTodos.map(item => {
                return (
                  <li
                  className = {item.finished ? "todo finished" : "todo"}
                  key = {item.id}
                  onClick = {() => finishTodo(item)}
```

```
                  onContextMenu = {e => delTodo(e,
item.id)}>{item.text}</li>
                )
              })
            }
          </ul>
        </form>
        <small>
          Left click if finished dotos
          <br/>
          Right click to delete
        </small>
    </div>
  );
}

export default App;
```

实验成绩：_____　　教师签名：_____